Projet INF5039 – Ingénierie Dirigée par les Modèles

ZOGO ABOUMA Zozime Achaire – 18N2824
NKOLO Stéphane Roylex – 21T2588

Université de Yaoundé 1 – Faculté des Sciences
Département d'Informatique – Master 2

Encadreur : Dr MONTHE Valérie

Année académique 2025–2026

Plan de la présentation

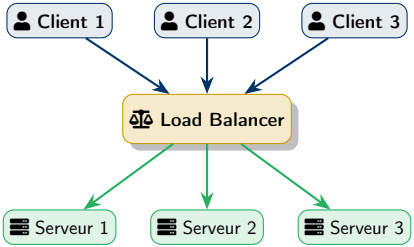
1. Domaine
2. Contexte et problématique
3. Objectifs visés
4. Solution proposée
5. Réalisations concrètes
6. Démonstration
7. Perspectives

Le domaine : la répartition de charge

Définition

La **répartition de charge** (*load balancing*) distribue le trafic réseau entre plusieurs serveurs pour :

- **Maximiser** le débit global
- **Minimiser** le temps de réponse
- **Éviter** la surcharge d'un seul nœud
- **Assurer** la haute disponibilité
- **Permettre** la scalabilité horizontale



Algorithmes de répartition

Algorithme	Principe	Usage
Round Robin	Tour à tour, distribution cyclique	Serveurs homogènes
Weighted RR	Round Robin avec poids par serveur	Capacités différentes
Least Connections	Vers le serveur le moins chargé	Requêtes longues
IP Hash	Hash de l'IP client	Affinité de session
Random	Choix aléatoire	Tests, cas simples

Notre DSL supporte **tous ces algorithmes** de manière déclarative

Le problème actuel

Limites de l'approche manuelle

- Configuration **spécifique** à chaque outil
- Syntaxes **différentes** (nginx.conf $\neq$ haproxy.cfg)
- **Vendor lock-in**
- Erreurs de syntaxe **difficiles à détecter**
- **Pas de validation** avant déploiement
- **Non portable**

Notre solution IDM

- Un DSL **unique**, indépendant
- Validation **formelle** (OCL)
- Génération **automatique** multi-cible
- **Portabilité** totale
- **Abstraction** du domaine
- **Réutilisabilité** des modèles

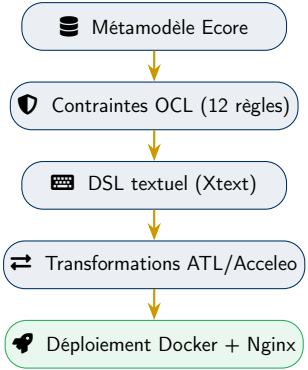
Nos objectifs

Objectif principal

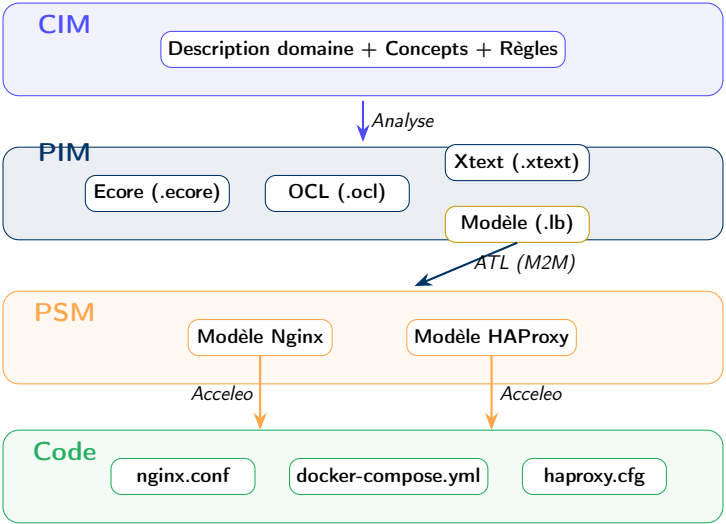
Concevoir un DSL pour modéliser la répartition de charge et **générer automatiquement** le code d'infrastructure via une approche MDA.

Objectifs spécifiques

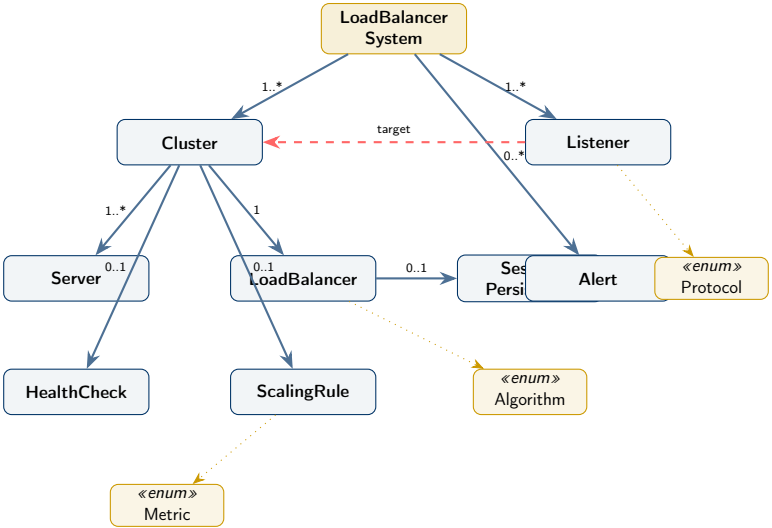
1. Définir le **métamodèle** du domaine
2. Formaliser les **contraintes** (OCL)
3. Créer la **syntaxe concrète** (Xtext)
4. Implémenter les **transformations**



Architecture MDA de notre approche



Le métamodèle (vue globale)



12 concepts – 12 contraintes OCL

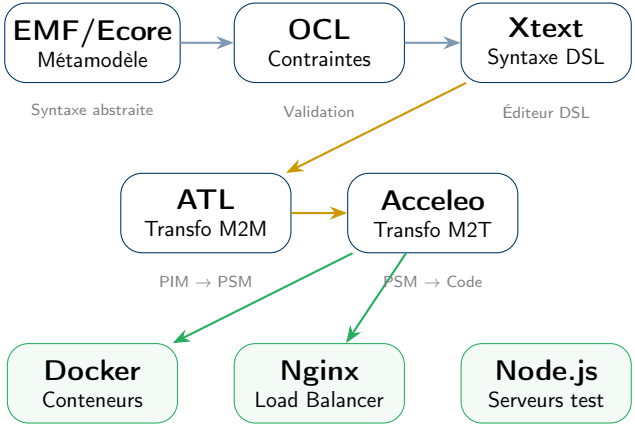
Concepts identifiés (12)

- LoadBalancerSystem
- Cluster
- Server
- LoadBalancer
- Algorithm *(enum)*
- HealthCheck
- Listener
- Protocol *(enum)*
- SessionPersistence
- Alert
- ScalingRule
- Metric *(enum)*

Contraintes OCL (12)

- ✓ Noms de serveurs uniques
- ✓ Port entre 1 et 65535
- ✓ Poids ≥ 1
- ✓ Cluster non vide
- ✓ Interval $>$ timeout
- ✓ ScaleUp $>$ scaleDown
- ✓ Min $\leq$ max instances
- ✓ Ports listeners uniques
- ✓ Weighted $\Rightarrow$ poids requis
- ✓ Cookie $\Rightarrow$ nom requis
- ✓ Au moins 1 listener
- ✓ Au moins 1 serveur actif

Technologies utilisées



La syntaxe DSL – Exemple concret

```
LoadBalancerSystem "ECommerceSystem" {
  description: "Systeme de repartition pour e-commerce"
  version: "1.0"

  cluster WebFrontend {
    loadbalancer {
      algorithm: ROUND_ROBIN
      stickySession: true
      sessionPersistence { type: COOKIE  cookieName: "SESSIONID"  ttl: 3600 }
    }
    server web1 { host: "web1"  port: 8080  weight: 3  enabled: true }
    server web2 { host: "web2"  port: 8080  weight: 2  enabled: true }
    server web3 { host: "web3"  port: 8080  weight: 1  enabled: true }
    healthCheck { protocol: HTTP  path: "/health"  interval: 30  timeout: 10 }
  }

  cluster APIBackend {
    loadbalancer { algorithm: LEAST_CONNECTIONS  stickySession: false }
    server api1 { host: "api1"  port: 3000  weight: 1  enabled: true }
    server api2 { host: "api2"  port: 3000  weight: 1  enabled: true }
    healthCheck { protocol: HTTP  path: "/api/health"  interval: 15  timeout: 5 }
  }
```

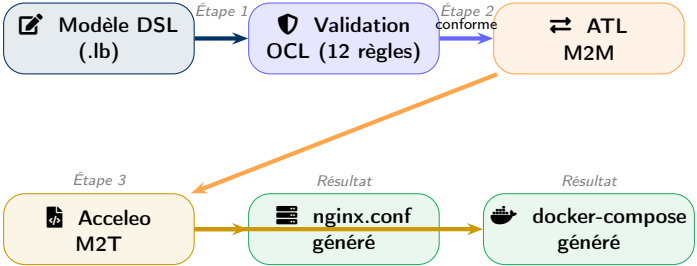
Code généré – nginx.conf

```
worker_processes auto;

events {
    worker_connections 1024;
}

http {
    upstream WebFrontend {
        server web1:8080 weight=3;
        server web2:8080 weight=2;
        server web3:8080 weight=1;
    }

    upstream APIBackend {
        least_conn;
        server api1:3000 weight=1;
        server api2:3000 weight=1;
    }

    server {
        listen 80;
        location / { proxy_pass http://WebFrontend; }
```

Code généré – docker-compose.yml

```yaml
services:
  web1:
    build: ./servers/web
    environment: [ SERVER_NAME=web1, SERVER_PORT=8080, SERVER_WEIGHT=3 ]
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
  web2:
    build: ./servers/web
    environment: [ SERVER_NAME=web2, SERVER_PORT=8080, SERVER_WEIGHT=2 ]
  web3:
    build: ./servers/web
    environment: [ SERVER_NAME=web3, SERVER_PORT=8080, SERVER_WEIGHT=1 ]
  api1:
    build: ./servers/api
    environment: [ SERVER_NAME=api1, SERVER_PORT=3000 ]
  api2:
    build: ./servers/api
    environment: [ SERVER_NAME=api2, SERVER_PORT=3000 ]

  loadbalancer:
```

Chaîne de transformations



T1 : PIM → PSM

ATL : LoadBalancer → Nginx Model

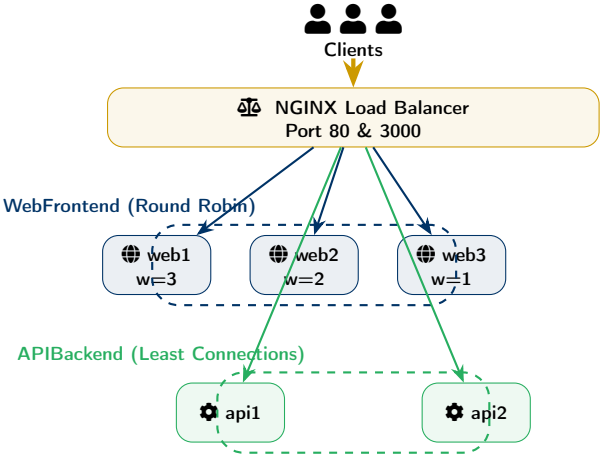
T3 : PSM → Code

Acceleo : Nginx → nginx.conf

T5 : PIM → Docker

Acceleo : Model → docker-compose

Architecture de la démo



Démo – Lancement

Commandes de déploiement

```
# 1. Se placer dans le dossier du cas d'etude
cd case-study/

# 2. Construire et demarrer tous les services
docker compose up --build -d

# 3. Verifier que tout est actif (6 conteneurs)
docker compose ps
```

Résultat attendu

web1	healthy	Serveur frontend (weight=3)
web2	healthy	Serveur frontend (weight=2)
web3	healthy	Serveur frontend (weight=1)
api1	healthy	Serveur API
api2	healthy	Serveur API

Démo – Test Round Robin

Test de la répartition (12 requêtes)

```
for i in $(seq 1 12); do
  curl -s http://localhost:80/ | python3 -c \
    "import sys,json; d=json.load(sys.stdin); \
     print(f'Req {$i}: {d[\"serverName\"]} (weight={d[\"weight\"]})')"
done
```

Distribution attendue (poids 3 :2 :1)

Démo – Test Failover

Simuler une panne

```
# Arreter web3
docker compose stop web3

# Les requetes continuent
for i in $(seq 1 6); do
  curl -s localhost:80/ \
  | python3 -c "import sys,json;\
  print(json.load(sys.stdin)\
  ['serverName'])"
done

# Redemarrer
docker compose start web3
```

Avant

- web1 ✔
- web2 ✔
- web3 ✔

→

Après

- web1 ✔
- web2 ✔
- web3 ✖

Le trafic est redistribué automatiquement !

Démo – Test de charge

Benchmark avec Apache Bench

```
# 1000 requetes, 50 en parallele
ab -n 1000 -c 50 http://localhost:80/

# Ou avec wrk (plus performant)
wrk -t4 -c100 -d30s http://localhost:80/
```

Métriques vérifiées

- Aucune requête perdue
- Distribution conforme aux poids
- Temps de réponse < 50ms
- Failover transparent

URLs de test

Frontend	`localhost:80`
API	`localhost:3000/api/products`
Health	`localhost:80/health`
Metrics	`localhost:80/metrics`

Ce qui resterait à faire

Court terme

- Support complet **HAProxy**
- Éditeur graphique avec **Eclipse Sirius**
- Génération de **manifestes Kubernetes**
- Support du **HTTPS/TLS**

Long terme

- Intégration **Prometheus + Grafana**
- Support **multi-cloud** (AWS, GCP, Azure)
- Load balancing **DNS**
- Interface **web** pour non-développeurs
- Validation par **simulation** de charge

L'approche IDM est extensible : chaque nouvelle plateforme cible ne nécessite qu'une **nouvelle transformation**, pas de refonte du DSL

Conclusion

Bilan du projet

- **DSL complet** pour la répartition de charge (12 concepts, grammaire Xtext)
- **12 contraintes OCL** formalisées et implémentées
- Architecture **MDA 4 niveaux** (CIM $\rightarrow$ PIM $\rightarrow$ PSM $\rightarrow$ Code)
- **Transformations** ATL (M2M) + Acceleo (M2T)
- **Cas d'étude réel** déployé avec Docker + Nginx

Modéliser une fois

$\Longrightarrow$ **Générer partout**

$\Longrightarrow$ **Déployer automatiquement**

Merci pour votre attention !

Questions ?

ZOGO ABOUMA Zozime Achaire – 18N2824
NKOLO Stéphane Roylex – 21T2588
Dr MONTHE Valérie
Université de Yaoundé 1
github.com/Achaire-Zogo/IDM-for-loadbalancing